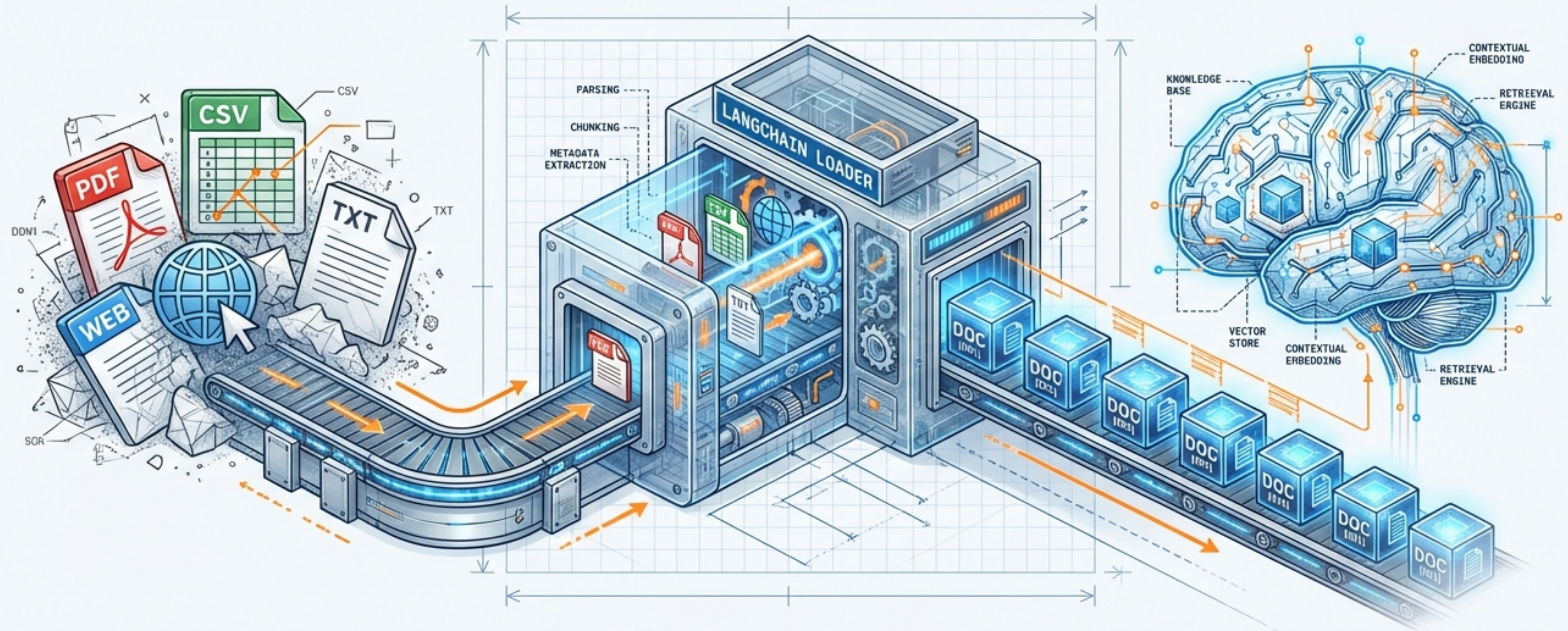
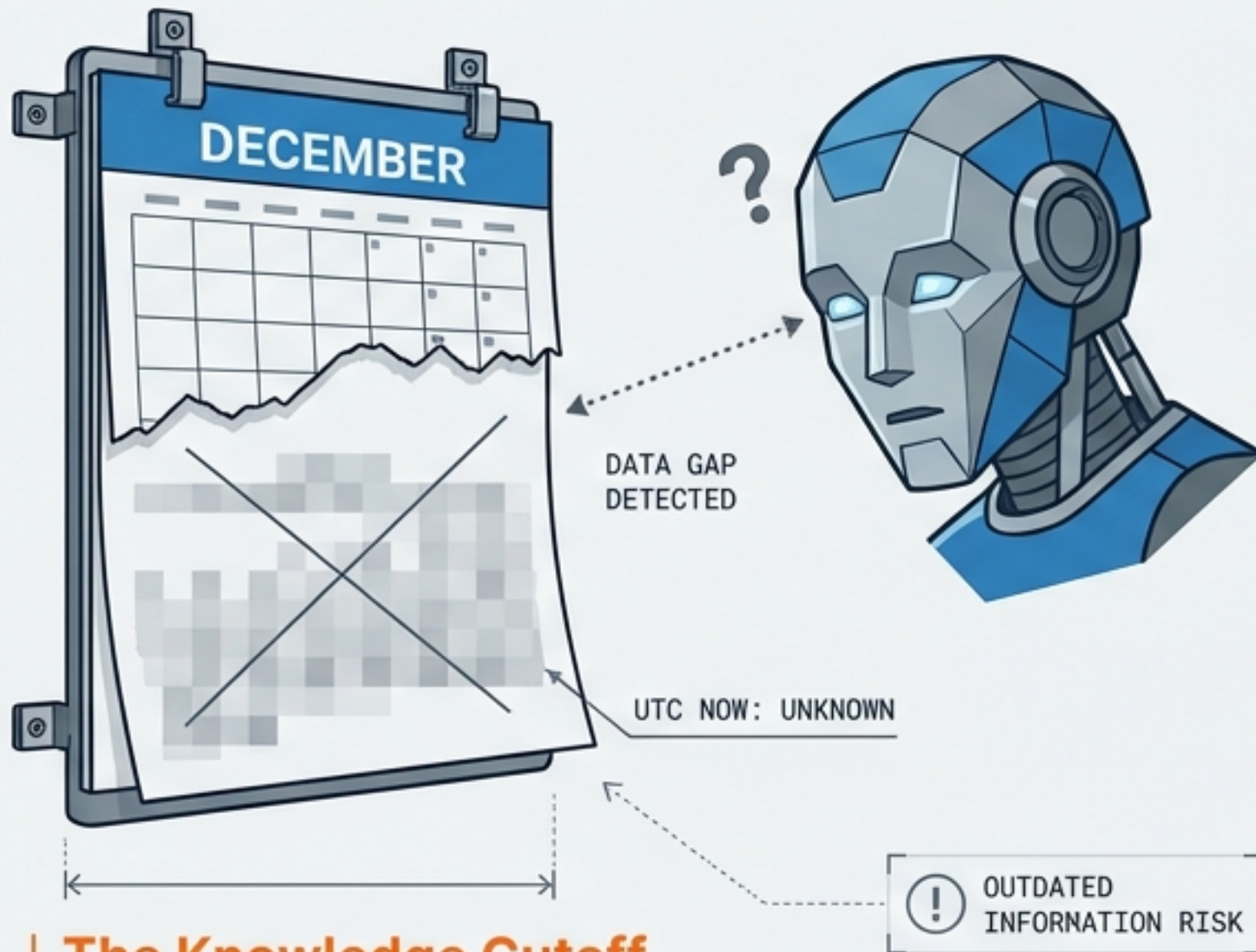


Mastering Document Loaders

The Architecture of Ingestion for RAG Applications



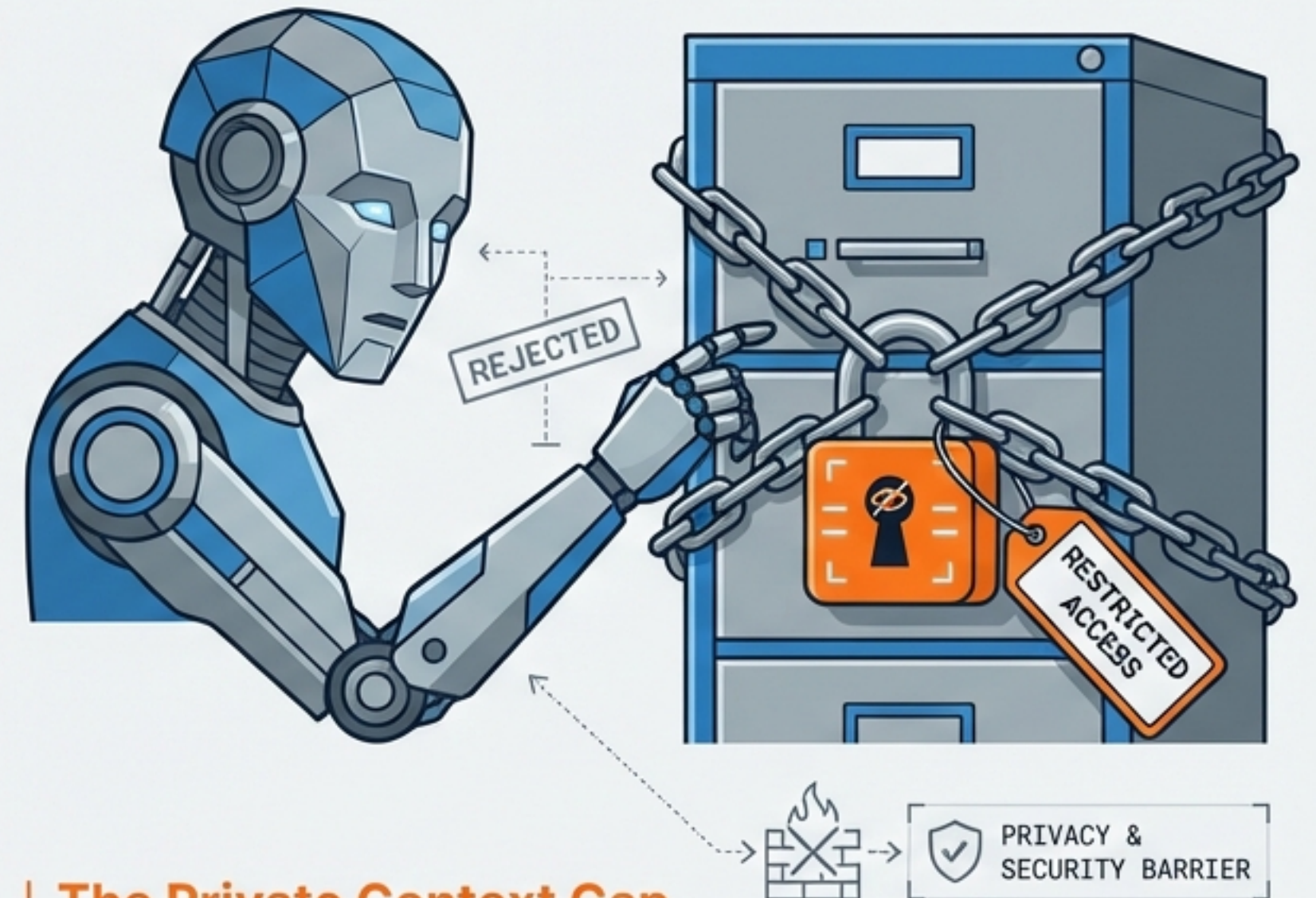
The Limitation of Standalone LLMs



The Knowledge Cutoff

LLMs are trained on historical data. They are unaware of current events, breaking news, or data generated today.

(Example: Latest market trends, live sensor readings, post-cutoff regulations)



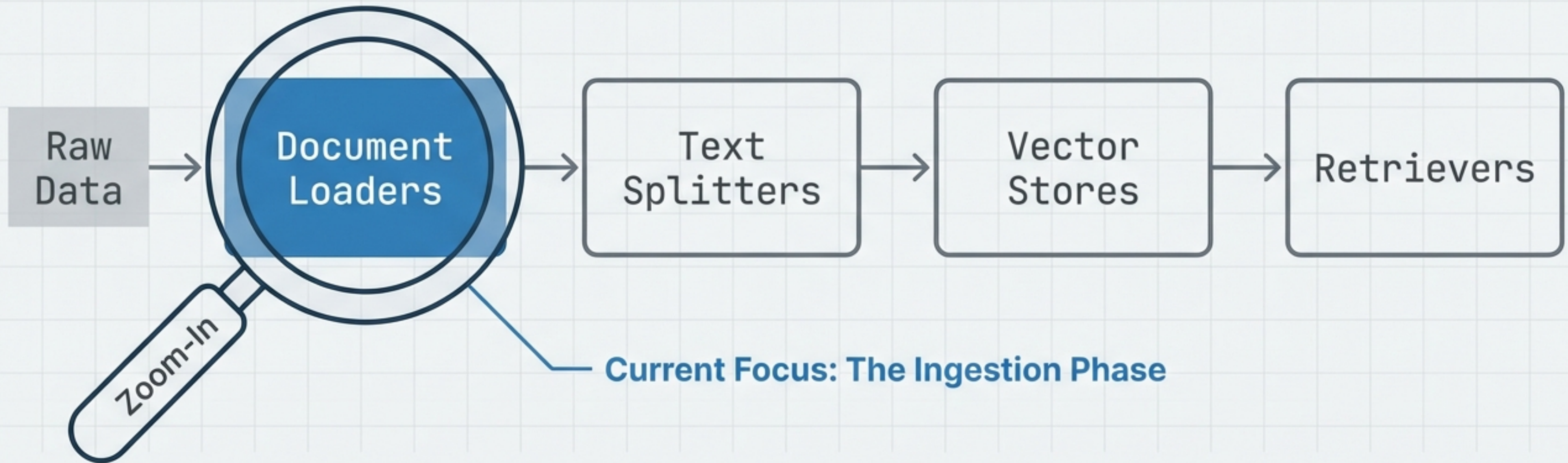
The Private Context Gap

LLMs do not know "you". They cannot access private emails, proprietary company documentation, internal databases, or user-specific information.

(Example: Internal project reports, CRM records, confidential communications)

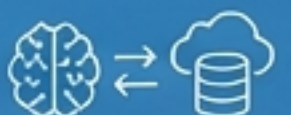
The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data

The RAG Architecture



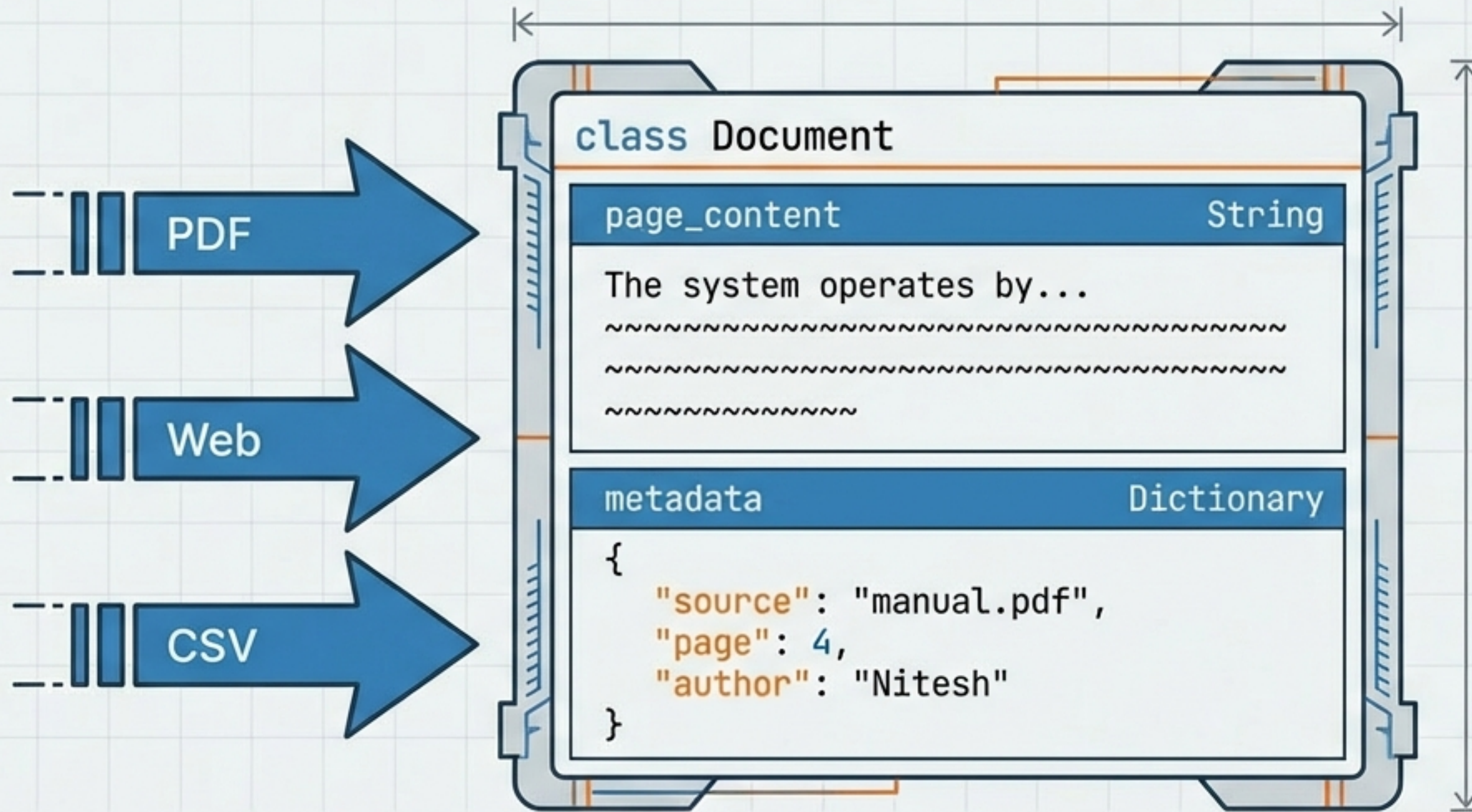
RAG combines information retrieval with language generation. It allows an LLM to cite sources it was never trained on. The process begins here: loading raw data from any source into the application.

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data

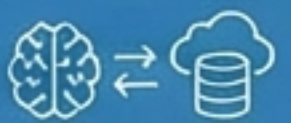


The Mechanism of Standardisation

Regardless of the input source, the output format in LangChain is always identical.



The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data

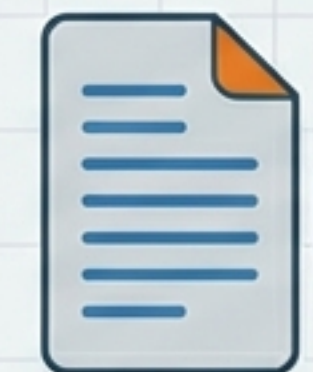


The Fundamental Loader: TextLoader

```
from langchain_community.document_loaders
import TextLoader
```

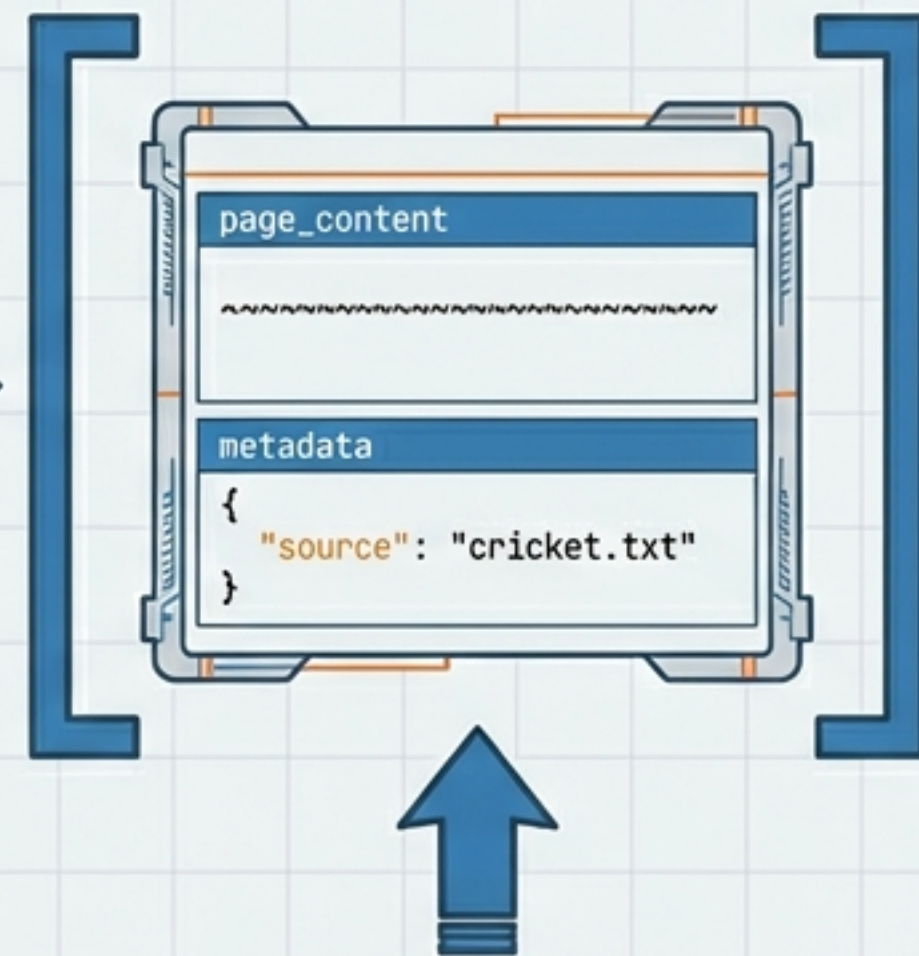
```
# Instantiate with path and encoding
loader = TextLoader('cricket.txt',
                    encoding='utf-8')
```

```
# Load into memory
docs = loader.load()
```



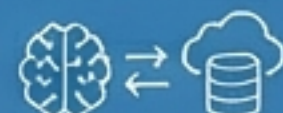
cricket.txt

.load()



Always returns a List,
even for single files.

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



Handling Business Documents: PyPDFLoader

Page-by-Page Loading Strategy

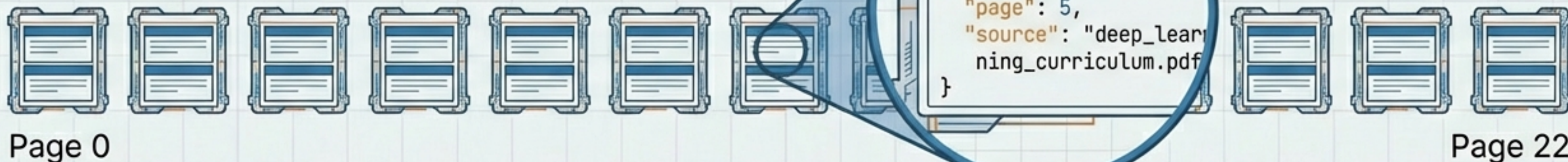
```
from langchain_community.document_loaders import PyPDFLoader

# Instantiate with the PDF file path
loader = PyPDFLoader('deep_learning_curriculum.pdf')

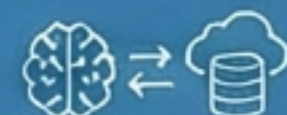
# Load the document, splitting by page
docs = loader.load()
```



deep_learning_curriculum.pdf



The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



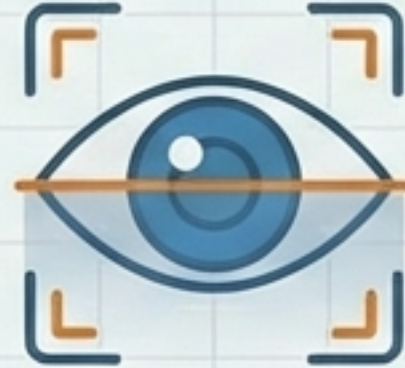
When PyPDF Isn't Enough: Advanced PDF Handling



Tables & Structure

PDFPlumberLoader

Best for extracting tabular data found in financial reports or invoices.



Scanned Images

UnstructuredPDFLoader

Uses OCR to extract text from non-selectable, image-based PDFs.

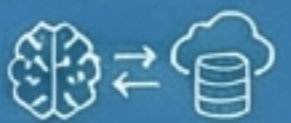


Enterprise Scale

AmazonTextractPDFLoader

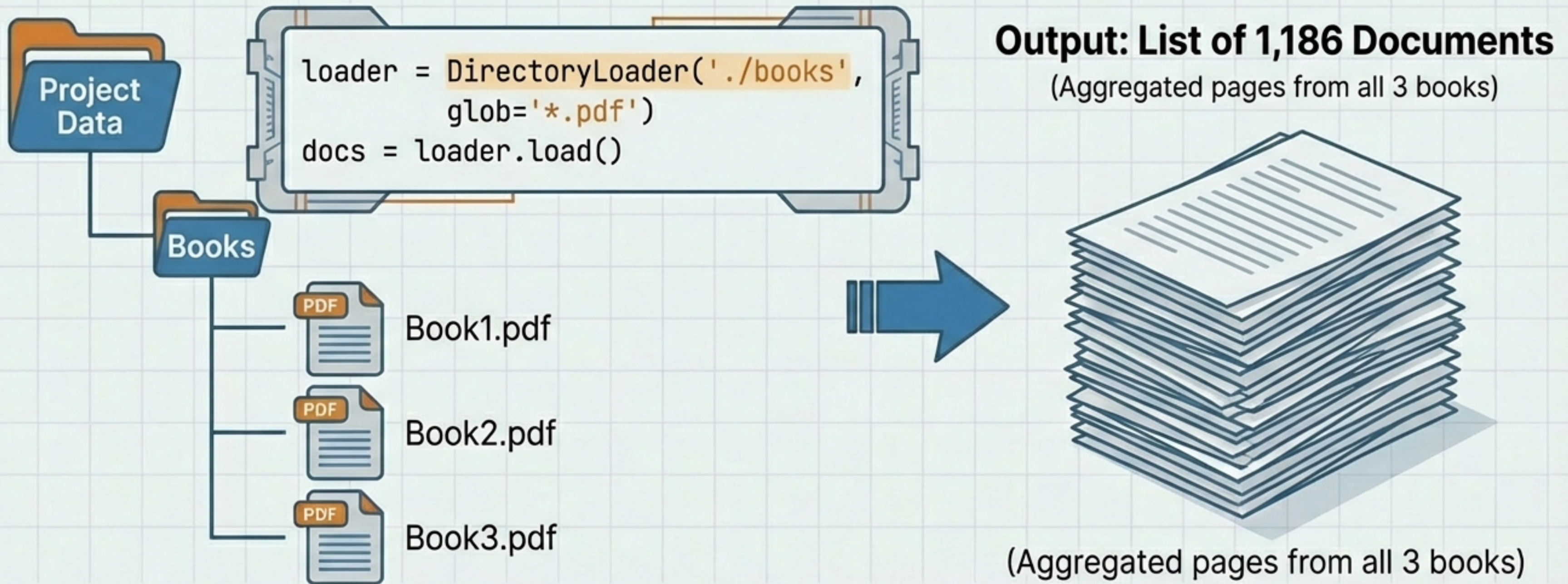
Cloud-native solution for high-volume, complex document processing.

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data

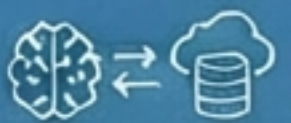


Bulk Ingestion with DirectoryLoader

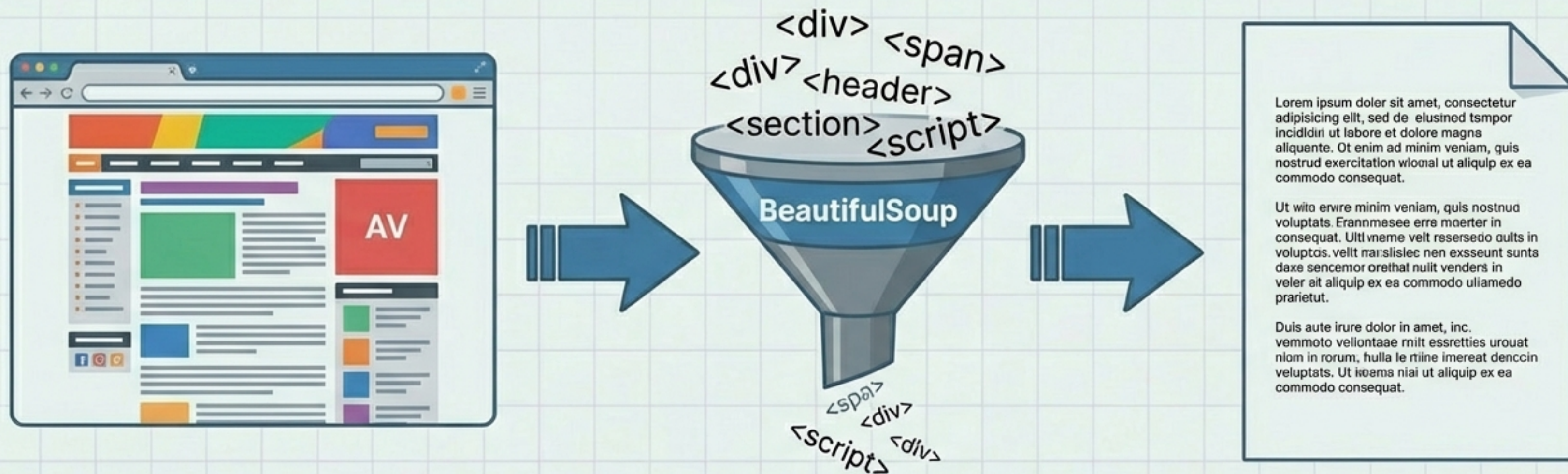
Ingest entire libraries using glob patterns to filter specific file types.



The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



Ingesting the Web: WebBaseLoader



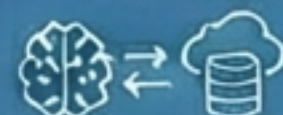
```
from langchain_community.document_loaders import WebBaseLoader
```

```
loader = WebBaseLoader('https://example.com/article')  
docs = loader.load()
```

Pro Tip

Note: For dynamic, JavaScript-heavy sites, use SeleniumURLLoader instead.

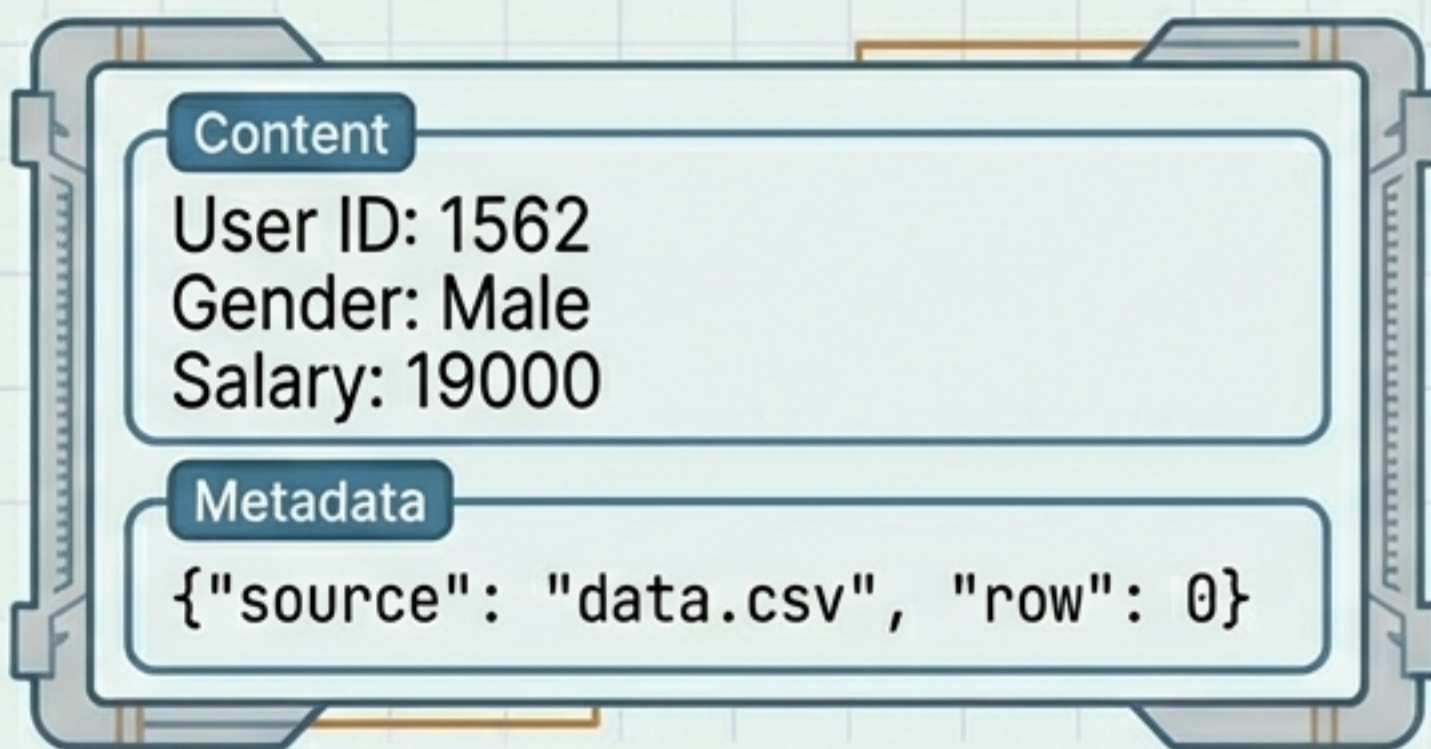
The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



Structured Data: CSVLoader

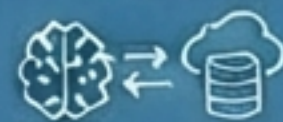
Maps individual rows to separate Document objects.

	User ID	Gender	Salary	
	1562	Male	19000	

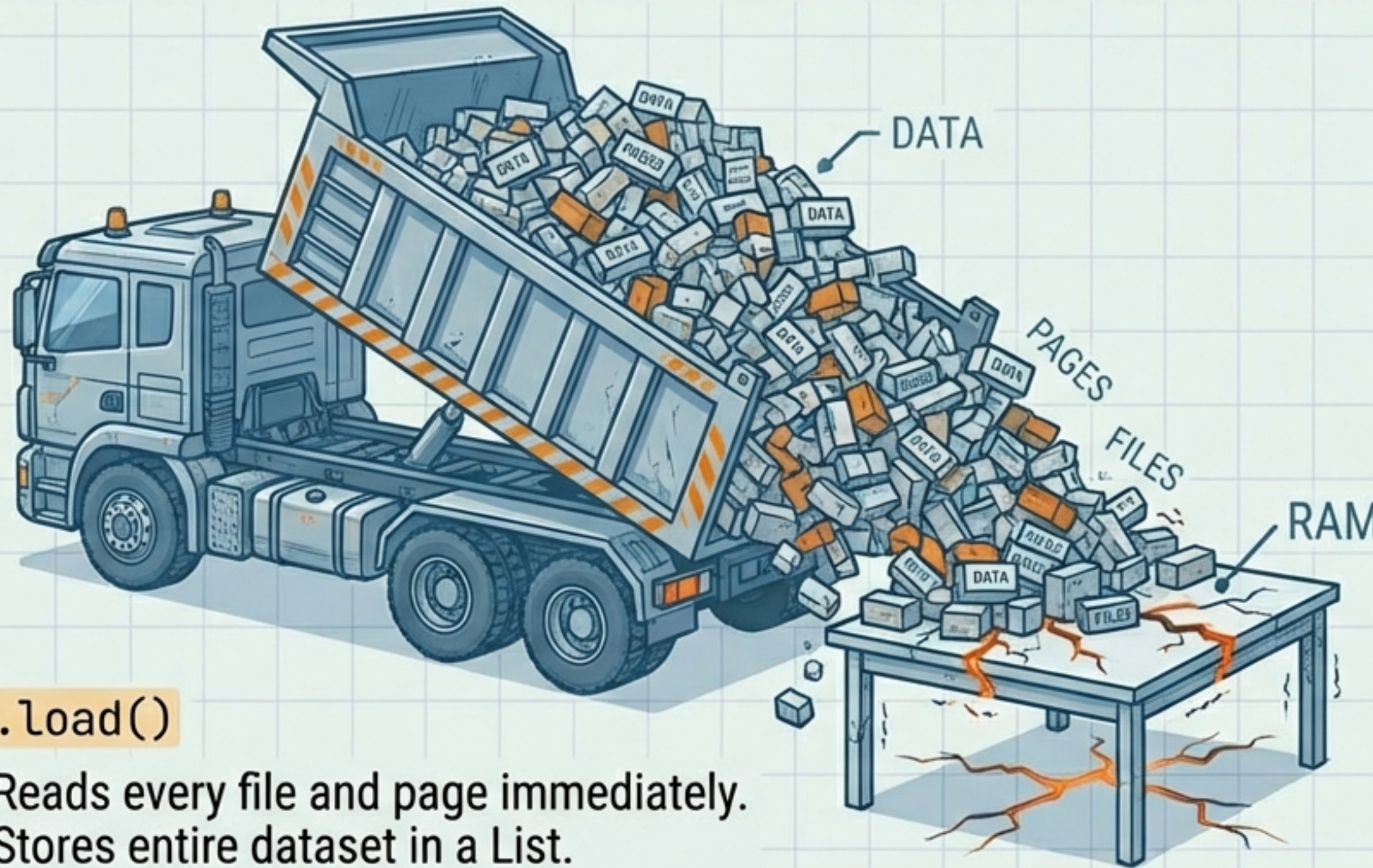


Enables Q&A over specific data entries (e.g., 'What is the salary of User 1562?').

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



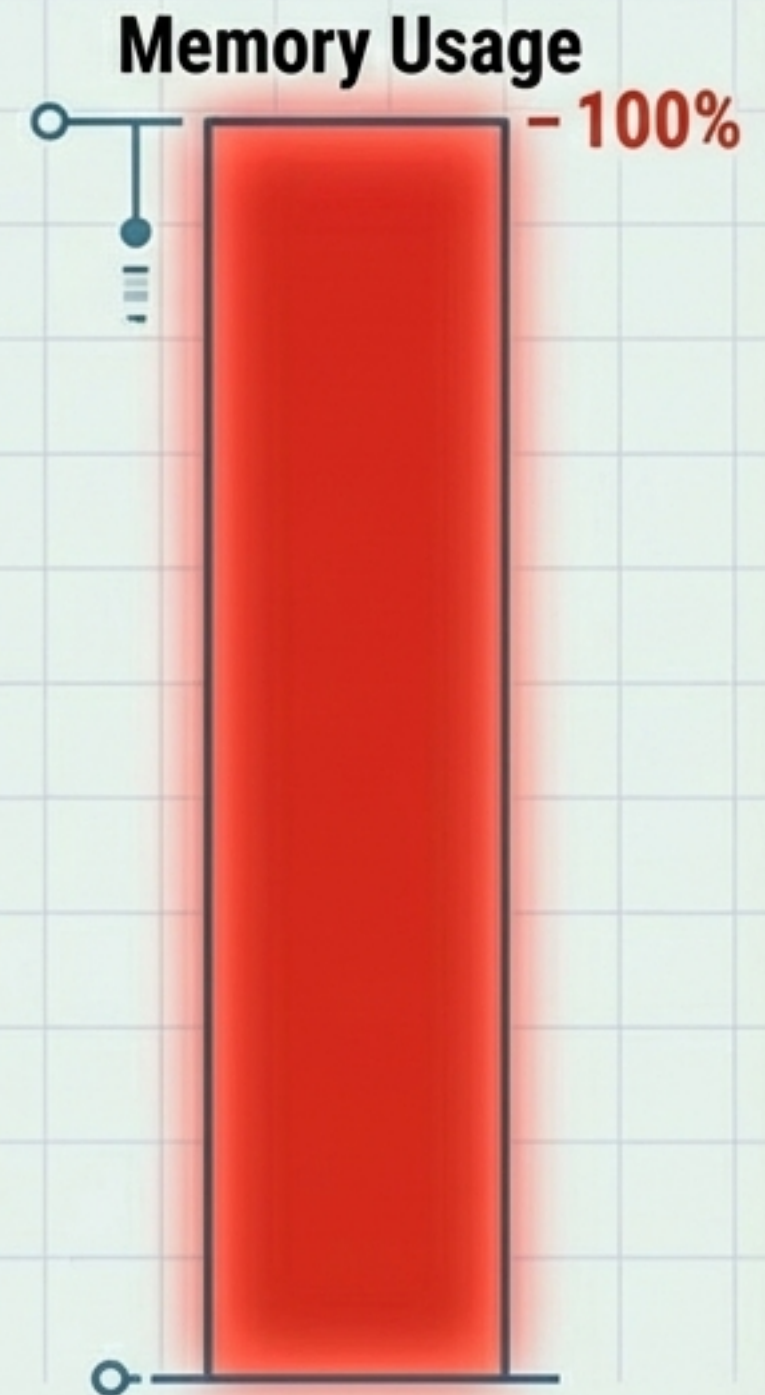
The Memory Bottleneck: Eager Loading



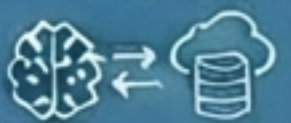
Method: `.load()`

Behavior: Reads every file and page immediately. Stores entire dataset in a List.

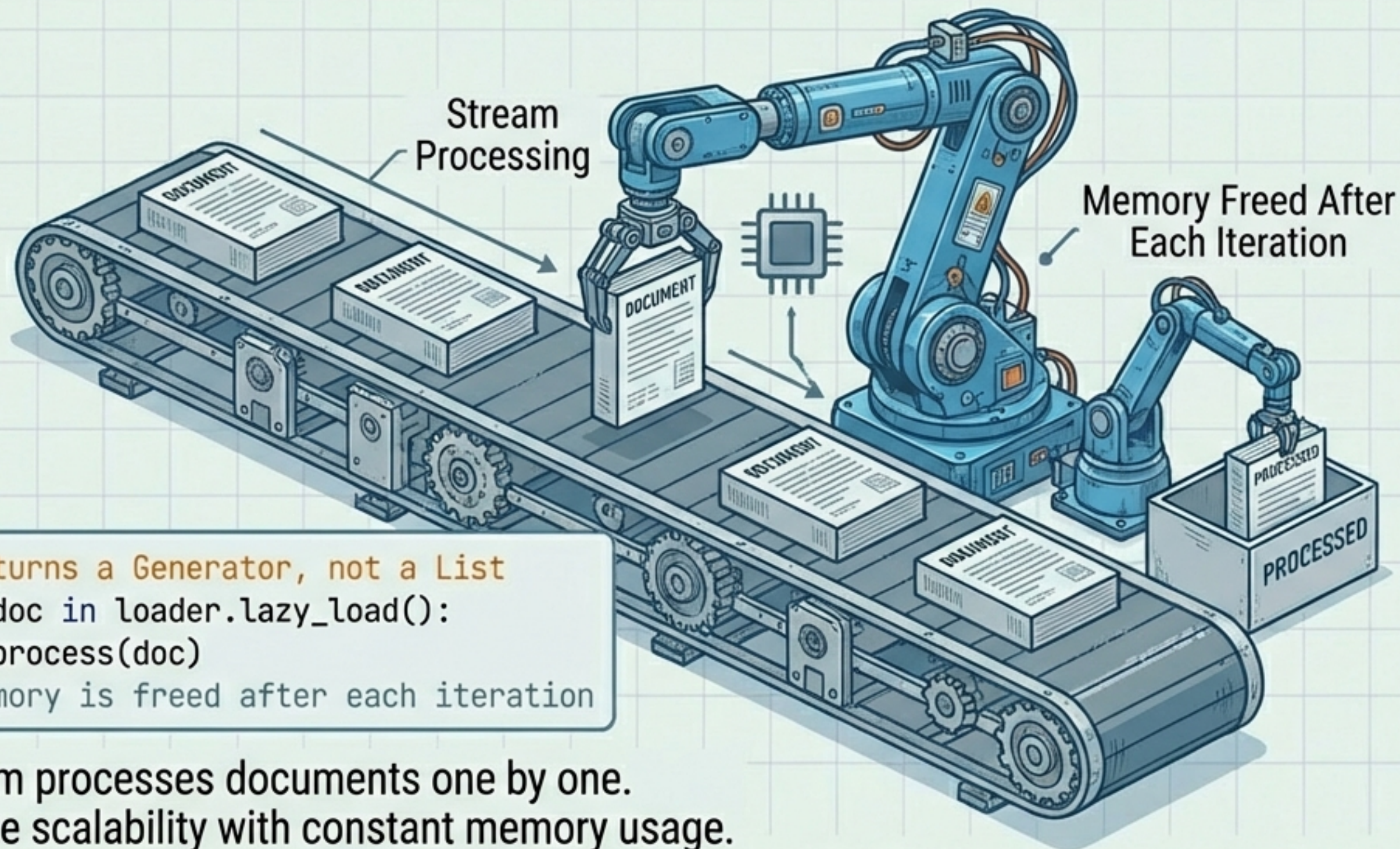
Result: Great for prototypes. Fatal for large production datasets.



The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



Optimising for Scale: Lazy Loading

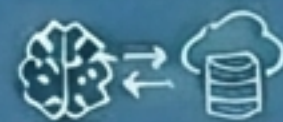


```
# Returns a Generator, not a List
for doc in loader.lazy_load():
    process(doc)
# Memory is freed after each iteration
```

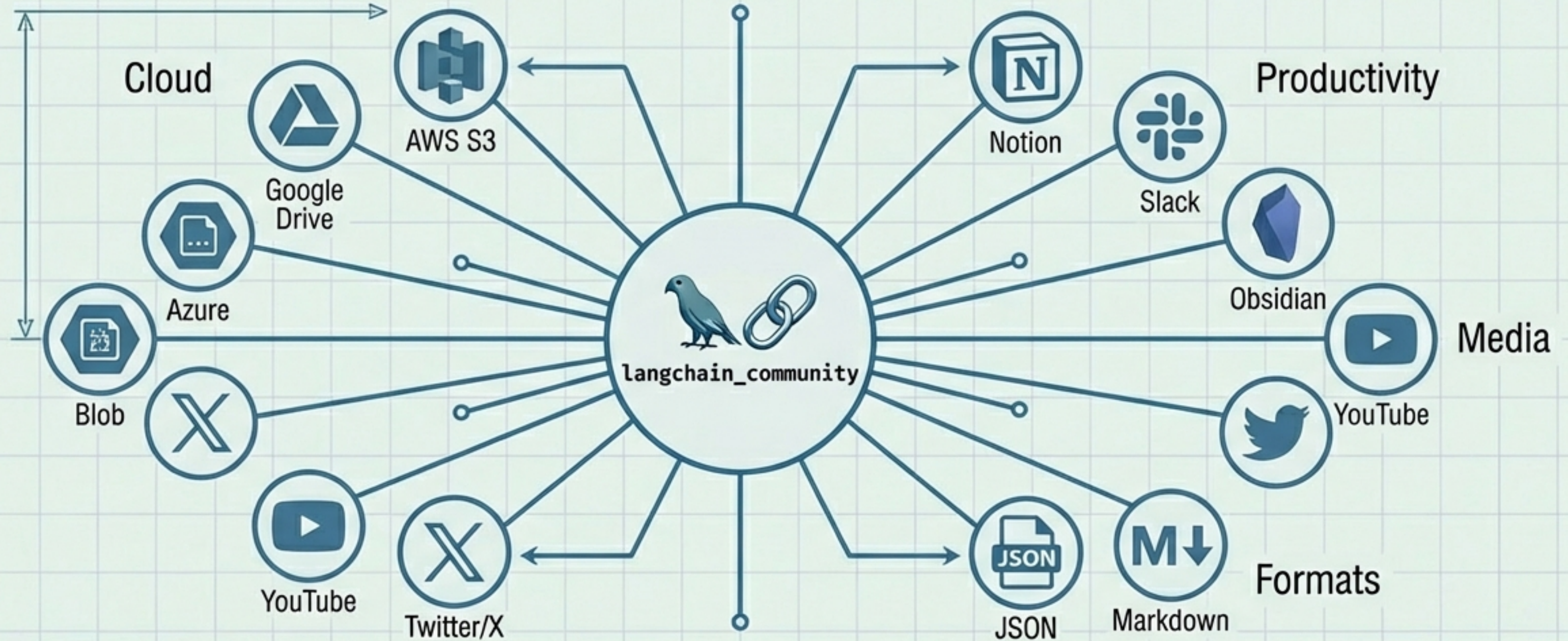
Stream processes documents one by one.
Infinite scalability with constant memory usage.



The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data

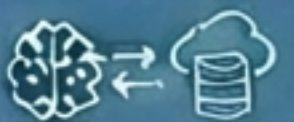


The LangChain Community Ecosystem

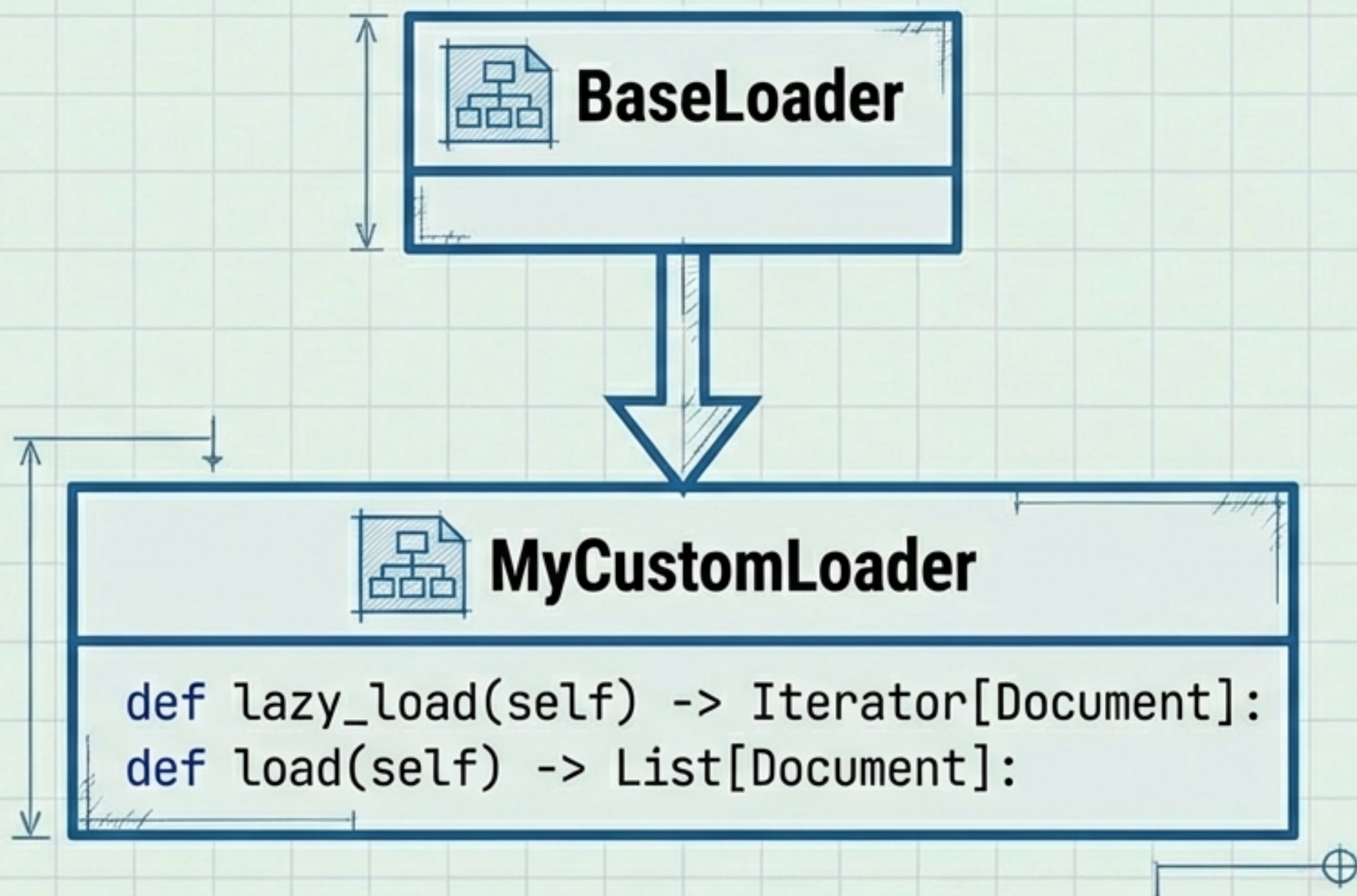


Hundreds of pre-built integrations are available instantly. You rarely need to build from scratch.

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data

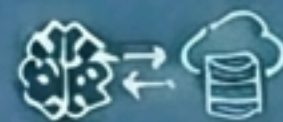


Beyond the Standard: Custom Loaders



Inherit from BaseLoader. Define your own logic to handle proprietary formats or internal APIs.

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



The Foundation of RAG is Set



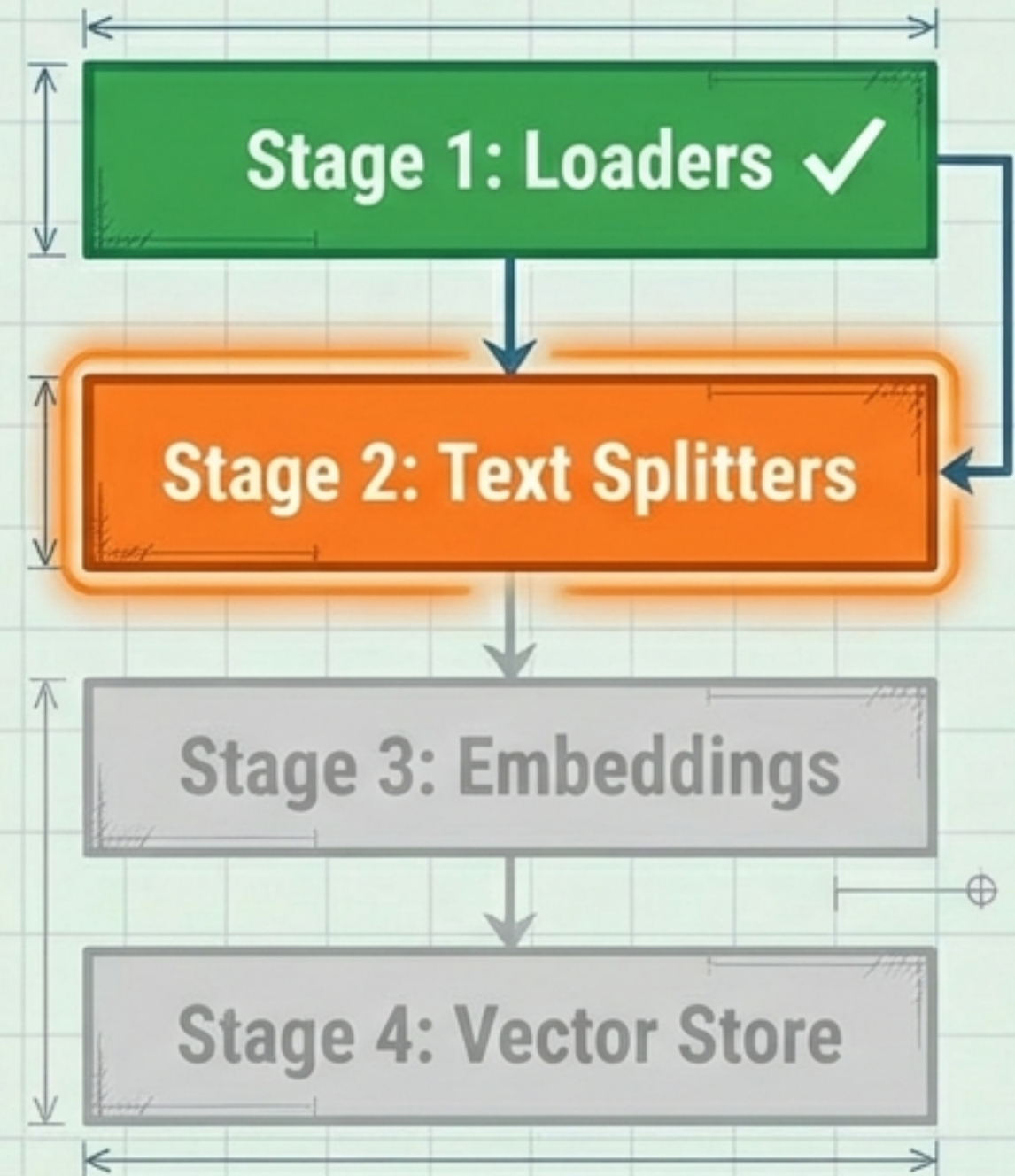
Ingest: Connected to Source (PDF, Web, CSV)



Standardise: Converted to Document Objects

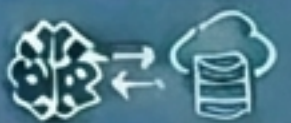


Optimise: Scaled with Lazy Loading



Next Module: Text Splitting & Chunking Strategies.

The Solution: **Retrieval Augmented Generation (RAG)** connects the LLM to external data



NotebookLM